

cpython/Lib/textwrap.py (PSF License)

subclasses to override in order to tweak the default behaviour.
If you want to completely replace the main wrapping algorithm,
you'll probably have to override `_wrap_chunks()`.

Several instance attributes control various aspects of wrapping:

```
width (default: 70)
    the maximum width of wrapped lines (unless break_long_words
    is false)
initial_indent (default: "")
    string that will be prepended to the first line of wrapped
    output. Counts towards the line's width.
subsequent_indent (default: "")
    string that will be prepended to all lines save the first
    of wrapped output; also counts towards each line's width.
expand_tabs (default: true)
    Expand tabs in input text to spaces before further processing.
    Each tab will become 0 .. 'tabsize' spaces, depending on its position
    in its line. If false, each tab is treated as a single character.
tabsize (default: 8)
    Expand tabs in input text to 0 .. 'tabsize' spaces, unless
    'expand_tabs' is false.
replace_whitespace (default: true)
    Replace all whitespace characters in the input text by spaces
    after tab expansion. Note that if expand_tabs is false and
    replace_whitespace is true, every tab will be converted to a
    single space!
fix_sentence_endings (default: false)
    Ensure that sentence-ending punctuation is always followed
    by two spaces. Off by default because the algorithm is
    (unavoidably) imperfect.
break_long_words (default: true)
    Break words longer than 'width'. If false, those words will not
    be broken, and some lines might be longer than 'width'.
break_on_hyphens (default: true)
    Allow breaking hyphenated words. If true, wrapping will occur
    preferably on whitespaces and right after hyphens part of
    compound words.
drop_whitespace (default: true)
    Drop leading and trailing whitespace from lines.
max_lines (default: None)
    Truncate wrapped lines.
placeholder (default: ' [...]')
    Append to the last line of truncated text.
"""
```

```
unicode_whitespace_trans = dict.fromkeys(map(ord, _whitespace), ord(' '))
```